

Main Packages and Class Codes

Package: com.sample.blogapplication

Class: BlogApplication

```
package com.sample.blogapplication;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BlogApplication {

    public static void main(String[] args) {
        SpringApplication.run(BlogApplication.class, args);
    }

}
```

Package: com.sample.blogapplication.controller

Class: CommentController

```
package com.sample.blogapplication.controller;

import com.sample.blogapplication.model.Comment;
import com.sample.blogapplication.model.User;
import com.sample.blogapplication.service.CommentService;
import com.sample.blogapplication.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/comments")
public class CommentController {

    @Autowired
    private CommentService commentService;

    @Autowired
    private UserService userService;

    @GetMapping("/post/{postId}")
    public ResponseEntity<List<Comment>> getCommentsByPostId(@PathVariable Long postId) {
        List<Comment> comments = commentService.getCommentsByPostId(postId);
        return ResponseEntity.ok(comments);
    }

    @PostMapping("/post/{postId}")
    public ResponseEntity<Comment> addComment(@PathVariable Long postId, @RequestBody Comment comment,
                                              @RequestHeader("username") String username) {
        User author = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        Comment createdComment = commentService.addComment(postId, comment, author);
        return ResponseEntity.status(HttpStatus.CREATED).body(createdComment);
    }

    @DeleteMapping("/{commentId}")
    public ResponseEntity<Void> deleteComment(@PathVariable Long commentId, @RequestHeader("username") String username) {
        User currentUser = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        commentService.deleteComment(commentId, currentUser);
        return ResponseEntity.noContent().build();
    }
}
```

Class: LikeController

```
package com.sample.blogapplication.controller;

import com.sample.blogapplication.model.User;
import com.sample.blogapplication.service.PostLikeService;
import com.sample.blogapplication.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/likes")
public class LikeController {

    @Autowired
    private PostLikeService postLikeService;

    @Autowired
    private UserService userService;

    @PostMapping("/post/{postId}/toggle")
    public ResponseEntity<Void> toggleLike(@PathVariable Long postId, @RequestHeader("username") String username) {
        User user = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        postLikeService.toggleLike(postId, user);
        return ResponseEntity.status(HttpStatus.OK).build();
    }

    @GetMapping("/post/{postId}/count")
    public ResponseEntity<Long> getLikeCount(@PathVariable Long postId) {
        long count = postLikeService.getLikeCount(postId);
        return ResponseEntity.ok(count);
    }
}
```

Class: PostController

```
package com.sample.blogapplication.controller;

import com.sample.blogapplication.model.Post;
import com.sample.blogapplication.model.User;
import com.sample.blogapplication.service.PostService;
import com.sample.blogapplication.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/posts")
public class PostController {

    @Autowired
    private PostService postService;

    @Autowired
    private UserService userService;

    @GetMapping
    public ResponseEntity<List<Post>> getAllPosts() {
        List<Post> posts = postService.getAllPosts();
        return ResponseEntity.ok(posts);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Post> getPostById(@PathVariable Long id) {
        Post post = postService.getPostById(id);
        return ResponseEntity.ok(post);
    }

    @PostMapping
    public ResponseEntity<Post> createPost(@RequestBody Post post, @RequestHeader("username") String username) {
        User author = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        Post createdPost = postService.createPost(post, author);
        return ResponseEntity.status(HttpStatus.CREATED).body(createdPost);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Post> updatePost(@PathVariable Long id, @RequestBody Post updatedPost,
                                           @RequestHeader("username") String username) {
        User currentUser = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        Post updated = postService.updatePost(id, updatedPost, currentUser);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletePost(@PathVariable Long id, @RequestHeader("username") String username) {
        User currentUser = userService.getUserByUsername(username)
            .orElseThrow(() -> new RuntimeException("User not found with username: " + username));
        postService.deletePost(id, currentUser);
        return ResponseEntity.noContent().build();
    }
}
```

Class: UserController

```
package com.sample.blogapplication.controller;

import com.sample.blogapplication.model.User;
import com.sample.blogapplication.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping("/register")
    public ResponseEntity<User> registerUser(@RequestBody User user) {
        User registeredUser = userService.registerUser(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(registeredUser);
    }
}
```

Package: com.sample.blogapplication.model

Class: Comment

```
package com.sample.blogapplication.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;

@Entity
@Table(name = "comments")
@Data
@NoArgsConstructor
@AllArgsConstructor

public class Comment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String content;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "author_id")
    private User author;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id")
    private Post post;

    @Column(name="created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name="updated_at", nullable = false)
    private LocalDateTime updated;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updated = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updated = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public User getAuthor() { return author; }
    public void setAuthor(User author) { this.author = author; }

    public Post getPost() { return post; }
    public void setPost(Post post) { this.post = post; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public LocalDateTime getUpdated() { return updated; }
    public void setUpdated(LocalDateTime updated) { this.updated = updated; }
}
```

Class: Post

```
package com.sample.blogapplication.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.List;

@Entity
@Table(name = "posts")
@Data
@AllArgsConstructor
@NoArgsConstructor

public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String content;

    @ManyToOne
    @JoinColumn(name = "author_id")
    private User author;

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Comment> comments;

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<PostLike> likes;

    @Column(name="created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name="updated_at", nullable = false)
    private LocalDateTime updated;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updated = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updated = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public User getAuthor() { return author; }
    public void setAuthor(User author) { this.author = author; }

    public List<Comment> getComments() { return comments; }
    public void setComments(List<Comment> comments) { this.comments = comments; }

    public List<PostLike> getLikes() { return likes; }
    public void setLikes(List<PostLike> likes) { this.likes = likes; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public LocalDateTime getUpdated() { return updated; }
    public void setUpdated(LocalDateTime updated) { this.updated = updated; }
}
```

Class: PostLike

```
package com.sample.blogapplication.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "post_likes" , uniqueConstraints = {
    @UniqueConstraint(columnNames = {"user_id", "post_id"})})
@Data
@NoArgsConstructor
@AllArgsConstructor
public class PostLike {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name="user_id", nullable = false)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name="post_id", nullable = false)
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    private Post post;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }

    public Post getPost() { return post; }
    public void setPost(Post post) { this.post = post; }
}
```

Class: Role

```
package com.sample.blogapplication.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "roles")
@Data
@NoArgsConstructor
@AllArgsConstructor

public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Class: User

```
package com.sample.blogapplication.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.util.Set;

@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor

public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false, unique = true)
    private String email;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Set<Role> getRoles() { return roles; }
    public void setRoles(Set<Role> roles) { this.roles = roles; }
}
```

Package: com.sample.blogapplication.repository

Class: CommentRepo

```
package com.sample.blogapplication.repository;

import com.sample.blogapplication.model.Comment;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface CommentRepo extends JpaRepository<Comment, Long> {

    List<Comment> findByPostIdOrderByCreatedAtDesc(Long postId);
}
```

Class: PostLikeRepo

```
package com.sample.blogapplication.repository;

import com.sample.blogapplication.model.PostLike;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface PostLikeRepo extends JpaRepository<PostLike, Long> {
    Optional<PostLike> findByUserIdAndPostId(Long userId, Long postId);
    long countByPostId(Long postId);
}
```

Class: PostRepo

```
package com.sample.blogapplication.repository;

import com.sample.blogapplication.model.Post;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface PostRepo extends JpaRepository<Post, Long> {
    List<Post> findByAuthorIdOrderByCreatedAtDesc(Long userId);
}
```

Class: RoleRepo

```
package com.sample.blogapplication.repository;

import com.sample.blogapplication.model.Role;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface RoleRepo extends JpaRepository<Role, Long> {
    Optional<Role> findByName(String name);
}
```

Class: UserRepo

```
package com.sample.blogapplication.repository;

import com.sample.blogapplication.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepo extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);

    Boolean existsByUsername(String username);
    Boolean existsByEmail(String email);
}
```

Package: com.sample.blogapplication.service

Class: CommentService

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.Comment;
import com.sample.blogapplication.model.User;

import java.util.List;

public interface CommentService {
    Comment addComment(Long id, Comment comment, User user);
    void deleteComment(Long commentId, User user);
    List<Comment> getCommentsByPostId(Long postId);
}
```

Class: CommentServiceImpl

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.Comment;
import com.sample.blogapplication.model.Post;
import com.sample.blogapplication.model.User;
import com.sample.blogapplication.repository.CommentRepo;
import com.sample.blogapplication.repository.PostRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class CommentServiceImpl implements CommentService {
    @Autowired
    private CommentRepo commentRepo;

    @Autowired
    private PostRepo postRepo;

    @Override
    public List<Comment> getCommentsByPostId(Long id) {
        return commentRepo.findByPostIdOrderByCreatedAtDesc(id);
    }

    @Override
    public Comment addComment(Long id, Comment comment, User user) {
        Post post=postRepo.findById(id)
            .orElseThrow(() -> new RuntimeException("Post not found with id: " + id));
        comment.setAuthor(user);
        comment.setPost(post);
        return commentRepo.save(comment);
    }

    @Override
    public void deleteComment(Long commentId, User user) {
        Comment comment = commentRepo.findById(commentId)
            .orElseThrow(() -> new RuntimeException("Comment not found with id: " + commentId));
        boolean isAdmin = user.getRoles().stream()
            .anyMatch(role -> role.getName().equals("ROLE_ADMIN"));
        boolean isAuthor = comment.getAuthor().getId().equals(user.getId());
        if(isAdmin || isAuthor){
            commentRepo.delete(comment);
        } else {
            throw new RuntimeException("You do not have permission to delete this comment.");
        }
    }
}
```

Class: PostLikeService

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.User;

public interface PostLikeService {
    void toggleLike(Long postId, User user);
    long getLikeCount(Long postId);
    boolean hasUserLikedPost(Long postId, User user);
}
```

Class: PostLikeServiceImp

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.Post;
import com.sample.blogapplication.model.PostLike;
import com.sample.blogapplication.model.User;
import com.sample.blogapplication.repository.PostLikeRepo;
import com.sample.blogapplication.repository.PostRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class PostLikeServiceImp implements PostLikeService {

    @Autowired
    private PostRepo postRepo;
    @Autowired
    private PostLikeRepo postLikeRepo;

    @Override
    public void toggleLike(Long postId, User user) {
        Post post = postRepo.findById(postId) // ✓ Use PostRepo
            .orElseThrow(() -> new RuntimeException("Post not found with id: " + postId));
        Optional<PostLike> postLikeOptional = postLikeRepo.findByIdAndPostId(user.getId(), postId);
        if(postLikeOptional.isPresent()) // ✓ Use correct variable name
        {
            postLikeRepo.delete(postLikeOptional.get());
        } else {
            PostLike newLike = new PostLike();
            newLike.setPost(post);
            newLike.setUser(user);
            postLikeRepo.save(newLike);
        }
    }

    @Override
    public long getLikeCount(Long postId) {
        return postLikeRepo.countByPostId(postId);
    }

    @Override
    public boolean hasUserLikedPost(Long postId, User user) {
        return postLikeRepo.findByIdAndPostId(user.getId(), postId).isPresent();
    }
}
```

Class: PostService

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.Post;
import com.sample.blogapplication.model.User;

import java.util.List;

public interface PostService {
    Post createPost(Post post, User user);
    Post updatePost(Long postId, Post updatedDetails, User currentUser);
    void deletePost(Long postId, User user);
    Post getPostById(Long id);
    List<Post> getAllPosts();
}
```

Class: PostServiceImp

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.Post;
import com.sample.blogapplication.model.User;
import com.sample.blogapplication.repository.PostRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PostServiceImp implements PostService {
    @Autowired
    private PostRepo postRepo;

    @Override
    public Post createPost(Post post, User author) {
        post.setAuthor(author);
        return postRepo.save(post);
    }

    @Override
    public Post updatePost(Long id, Post UpdatedDetails, User currentUser) {
        Post existingPost = getPostById(id);
        boolean isAdmin = currentUser.getRoles().stream()
            .anyMatch(role -> role.getName().equals("ROLE_ADMIN"));
        boolean isAuthor = existingPost.getAuthor().getId().equals(currentUser.getId());
        if(isAdmin || isAuthor){
            existingPost.setTitle(UpdatedDetails.getTitle());
            existingPost.setContent(UpdatedDetails.getContent());
            return postRepo.save(existingPost);
        } else {
            throw new RuntimeException("You do not have permission to update this post.");
        }
    }

    @Override
    public void deletePost(Long id, User currentUser) {
        Post existingPost = getPostById(id);
        boolean isAdmin = currentUser.getRoles().stream()
            .anyMatch(role -> role.getName().equals("ROLE_ADMIN"));
        boolean isAuthor = existingPost.getAuthor().getId().equals(currentUser.getId());
        if(isAdmin || isAuthor){
            postRepo.delete(existingPost);
        } else {
            throw new RuntimeException("You do not have permission to delete this post.");
        }
    }

    @Override
    public Post getPostById(Long id) {
        return postRepo.findById(id)
            .orElseThrow(() -> new RuntimeException("Post not found with id: " + id));
    }

    @Override
    public List<Post> getAllPosts() {
        return postRepo.findAll();
    }
}
```

Class: UserService

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.User;

import java.util.Optional;

public interface UserService {
    User registerUser(User user);
    Optional<User> getUserByUsername(String username);
}
```

Class: UserServiceImp

```
package com.sample.blogapplication.service;

import com.sample.blogapplication.model.User;
import com.sample.blogapplication.repository.RoleRepo;
import com.sample.blogapplication.repository.UserRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class UserServiceImp implements UserService {

    @Autowired
    private UserRepo userRepo;

    @Override
    public User registerUser(User user) {
        if(userRepo.existsByUsername(user.getUsername())) {
            throw new RuntimeException("Username is already taken");
        }
        if(userRepo.existsByEmail(user.getEmail())) {
            throw new RuntimeException("Email is already in use");
        }

        return userRepo.save(user);
    }

    @Override
    public Optional<User> getUserByUsername(String username) {
        return userRepo.findByUsername(username);
    }
}
```